UNIVERSITY OF WESTMINSTER

INFORMATICS INSTITUTE OF TECHNOLOGY

Informatics Institute of Technology

School of Computing

Software Development II Coursework Report

Module : 4COSC010C.2: Software Development II (2023)

Date of submission : 15th July 2024 (6.00 AM)

Student ID : 20221804 / 20532233

Student First Name : Saad

Student Surname : Mazhar

Tutorial group : Group 22

"I confirm that I understand what plagiarism / collusion / contract cheating is and have read and understood the section on Assessment Offences in the Essential Information for Students. The work that I have submitted is entirely my own. Any work from other authors is duly referenced and acknowledged."

Name : Saad Mazhar

Student ID : 20221804

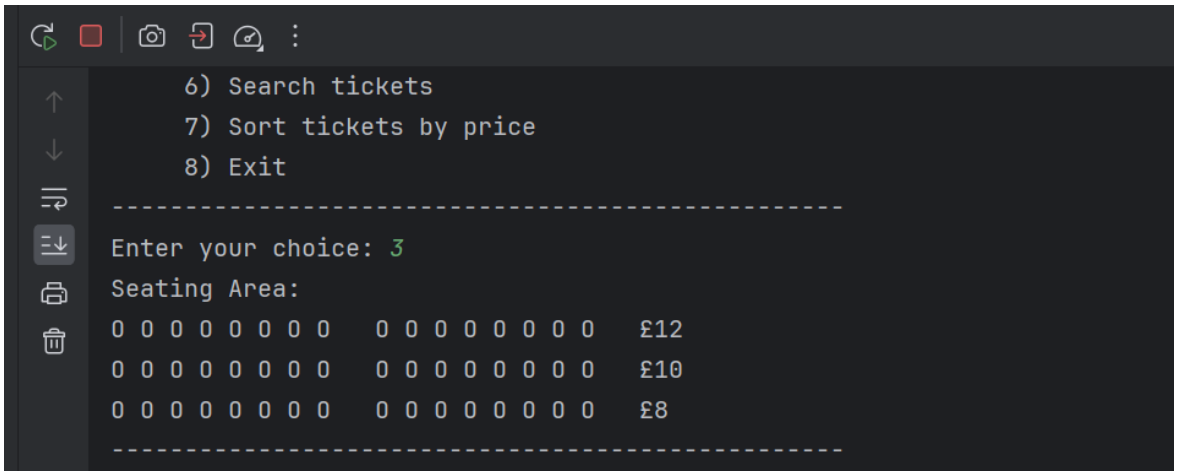
Self-assessment form and test plan

1) Self-assessment form

Task	Self-assessment (select one)	Comments
1	☒Fully implemented ☐Partially implemented ☐Not attempted	Project was titled as CinemaManagement. And the tasks were completed.
2	☒Fully implemented ☐Partially implemented ☐Not attempted	
Insert here a screenshot of your welcome message and menu:  		
3	☒Fully implemented ☐Partially implemented ☐Not attempted	
4	☒Fully implemented ☐Partially implemented ☐Not attempted	
5	☒Fully implemented ☐Partially implemented ☐Not attempted	

6	☒Fully implemented ☐Partially implemented ☐Not attempted	Has 16 seats per row. And a gap between seat 8 and 9. (as shown in the figure)

Insert here a screenshot of the seating plan:



7	☒Fully implemented ☐Partially implemented ☐Not attempted	Created Person class.
8	☒Fully implemented ☐Partially implemented ☐Not attempted	Created Ticket class.
9	☒Fully implemented ☐Partially implemented ☐Not attempted	
10	☒Fully implemented ☐Partially implemented ☐Not attempted	
11	☒Fully implemented ☐Partially implemented ☐Not attempted	
12	☒Fully implemented ☐Partially implemented ☐Not attempted	

2) Test Plan

Complete the test plan describing which testing you have performed on your program.
Add as many rows as you need.

Part A Testing

Test case / scenario	Input	Expected Output	Output	Pass/Fail
Buying a ticket for a valid seat.	Row = 2, Seat = 5, Name = "Saad", Surname = "Mazhar", Email = "saad.20221804@iit.ac.lk"	The seat has been booked.	The seat has been booked.	☒Pass ☐Fail
Attempting to buy a ticket for an invalid row.	Row = 6, Seat = 4, Name = "Saad", Surname = "Mazhar", Email = "saad.20221804@iit.ac.lk"	Invalid seat. Please try again.	Invalid seat. Please try again.	☒Pass ☐Fail
Attempting to buy a ticket for an invalid seat number.	Row = 2, Seat = 18, Name = "Saad", Surname = "Mazhar", Email = "saad.20221804@iit.ac.lk"	Invalid seat. Please try again.	Invalid seat. Please try again.	☒Pass ☐Fail
Cancelling a ticket for a valid seat.	Row = 2, Seat = 5	The seat has been cancelled.	The seat has been cancelled.	☒Pass ☐Fail
Attempting to cancel a ticket for an invalid row.	Row = 6, Seat = 5	Invalid seat. Please try again.	Invalid seat. Please try again.	☒Pass ☐Fail
Printing the seating area when no tickets have been sold.	From the main menu, select 3.	5 rows of 17 seats with all 'O' and the respective row prices.	5 rows of 17 seats with all 'O' and the respective row prices.	☒Pass ☐Fail
Printing the seating area after some tickets have been sold.	Buy a ticket for Row = 2, Seat = 5.	Row 2 should have 'X' at seat 5, rest as 'O', and respective row prices.	Row 2 should have 'X' at seat 5, rest as 'O', and respective row prices.	☒Pass ☐Fail
Finding the first available seat after some tickets have been sold.	Buy tickets for Row = 1, Seat = 1 and Row = 1, Seat = 2.	First available seat: Row 1 Seat 3	First available seat: Row 1 Seat 3	☒Pass ☐Fail

Finding the first available seat when all seats have been sold.	(Buy tickets for all seats.)	No available seats.	No available seats.	☒Pass ☐Fail
Sorting tickets by price after some tickets have been sold.	Buy tickets for Row = 1, Seat = 1 (£12), Row = 2, Seat = 1 (£10), Row = 3, Seat = 1 (£8).	Tickets sorted by price with details and total sales: £30.00.	Tickets sorted by price with details and total sales: £30.00.	☒Pass ☐Fail

Part B testing

Test case / scenario	Input	Expected Output	Output	Pass/Fail
Invalid Row Number on Buy Ticket	Row = 0, Seat = 5, Name = "Saad", Surname = "Mazhar", Email = "saad.20221804@iit.ac.lk"	Invalid seat. Please try again.	Invalid seat. Please try again.	☒Pass ☐Fail
Invalid Seat Number on Buy Ticket	Row = 3, Seat = 0, Name = "Saad", Surname = "Mazhar", Email = "saad.20221804@iit.ac.lk"	Invalid seat. Please try again.	Invalid seat. Please try again.	☒Pass ☐Fail
Non-numeric Input for Row	Row = abc, Seat = 0, Name = "Saad", Surname = "Mazhar", Email = "saad.20221804@iit.ac.lk"	Handle the exception and prompt for valid input.	Handle the exception and prompt for valid input.	☒Pass ☒Fail
Buy Last Available Seat	Row = 3, Seat = 12, Name = "Saad", Surname = "Mazhar", Email = "saad.20221804@iit.ac.lk"	The seat has been booked.	The seat has been booked.	☐Pass ☐Fail
				☐Pass ☐Fail
				☐Pass ☐Fail
				☐Pass ☐Fail

Are there any specific parts of the coursework which you would like to get feedback?

You will need to demonstrate your understanding of the submitted code. Your tutor will arrange a coursework demonstration. During the coursework demonstration, your tutor will ask you to execute your program and questions on your code.

Failure to attend the demonstration will result in 0 for the coursework.

3) Code :

Main Class

```
import java.util.Scanner; // Import the Scanner class for user input
import java.util.Arrays; // Import the Arrays class for sorting
import java.util.Comparator; // Import the Comparator class for custom
sorting

// Main class for cinema management
public class CinemaManagement {

    // Constants
    private static final int ROWS = 3; // Number of rows in the cinema
    private static final int SEATS_PER_ROW = 16; // Number of seats per row
    private static final int[] ROW_PRICES = {12, 10, 8}; // Prices for each
row

    // Seat status array: 0 means available, 1 means booked
    private static int[][] seats = new int[ROWS][SEATS_PER_ROW];
    private static Ticket[] tickets = new Ticket[ROWS * SEATS_PER_ROW]; //
Array to store sold tickets
    private static int ticketCount = 0; // Counter for the number of sold
tickets

    public static void main(String[] args) {
        System.out.println("-
-");
        System.out.println("--------- Welcome to The London Lumiere ---------
-"); //Welcome Message

        while (true) { // Infinite loop to keep the menu running
            Menu();
```

```
            int choice = User_Choice();
            switch (choice) { // Switch case based on user's choice
                case 1: //if user input is 1, go to buy ticket method
                    buy_Ticket();
                    break;
                case 2: //if user input is 2, go to cancel ticket method
                    cancel_Ticket();
                    break;
                case 3:  //if user input is 3, go to print_seating_area
method
                    print_seating_area();
                    break;
                case 4: //if user input is 4, go to find_first_available
method
                    find_first_available();
                    break;
                case 5: //if user input is 5, go to  print_tickets_info
method
                    print_tickets_info();
                    break;
                case 6: //if user input is 6, go to search_ticket method
                    search_tickets();
                    break;
                case 7: //if user input is 7, go to sort_tickets method
                    sort_tickets();
                    break;
                case 8: // If user enters 8 then Exit the program
                    System.out.println("Exiting the program. \n GoodBye!");
                    return;
                default:
                    System.out.println("Invalid choice. Please try again.");
            }
        }
    }
    private static void Menu() { // Method to display the menu
        System.out.println("-------------------------------------------------
-");
        System.out.println("-                   MENU OPTIONS
-");
        System.out.println("-------------------------------------------------
-");
        System.out.println("     1) Buy a ticket");
        System.out.println("     2) Cancel a ticket");
        System.out.println("     3) See Seating Plan");
        System.out.println("     4) Find First Seat Available");
        System.out.println("     5) Print ticket information and total
price");
        System.out.println("     6) Search tickets");
        System.out.println("     7) Sort tickets by price");
        System.out.println("     8) Exit");
        System.out.println("-------------------------------------------------
-");
    }
    private static int User_Choice() { //Method to get the users choice
        Scanner scanner = new Scanner(System.in); // Scanner for user input
        System.out.print("Enter your choice: ");
        return scanner.nextInt(); // Return the user's choice
```

```
    }

//    static void buy_ticket() {
//        while (true) {
//            System.out.println("Enter your preferred row (1 , 2 , 3)");
//ask user the preferred row
//            String row_number = user_input.next();
//
//            if (row_number.matches("[123]")) {    //check if input is
within the range (1, 2 or 3)
//                details_buy(row_number);    //call the details_buy
//
//                break;
//
//            } else {
//                System.out.println("Invalid row. Please enter 1 , 2 or 3");
//            }
//        }
//    }

    private static void buy_Ticket() { // Method to buy a ticket
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter row number (1-3): ");
        int row = scanner.nextInt() - 1;
        System.out.print("Enter seat number (1-16): ");
        int seat = scanner.nextInt() - 1;

        // Check if seat is valid
        if (row < 0 || row >= ROWS || seat < 0 || seat >= SEATS_PER_ROW) {
            System.out.println("Invalid seat. Please try again.");
        } else if (seats[row][seat] == 1) { // Check if seat is already sold
            System.out.println("This seat is not available.");
        } else {
            seats[row][seat] = 1;

            // Collect person information
            System.out.print("Enter your name: ");
            String name = scanner.next();
            System.out.print("Enter your surname: ");
            String surname = scanner.next();
            System.out.print("Enter your email: ");
            String email = scanner.next();

            // Create Person and Ticket objects
            Person person = new Person(name, surname, email); // Create a
Person object
            Ticket ticket = new Ticket(row, seat, ROW_PRICES[row], person);
// Create a Ticket object
            tickets[ticketCount++] = ticket;

            System.out.println("The seat has been booked.");
        }
    }

    private static void cancel_Ticket() {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter row number (1-3): ");
```

```
        int row = scanner.nextInt() - 1;
        System.out.print("Enter seat number (1-16): ");
        int seat = scanner.nextInt() - 1;

        if (row < 0 || row >= ROWS || seat < 0 || seat >= SEATS_PER_ROW) {
            System.out.println("Invalid seat. Please try again.");
        } else if (seats[row][seat] == 0) {
            System.out.println("This seat is already available.");
        } else {
            seats[row][seat] = 0;
            for (int i = 0; i < ticketCount; i++) {
                if (tickets[i].getRow() == row && tickets[i].getSeat() ==
seat) {
                    // Shift the remaining tickets left
                    System.arraycopy(tickets, i + 1, tickets, i, ticketCount
- i - 1);
                    ticketCount--; // Decrease the ticket count
                    break;
                }
            }
            System.out.println("The seat has been cancelled."); //
Confirmation message
        }
    }


    private static void print_seating_area() {
        System.out.println("Seating Area:");
        for (int i = 0; i < ROWS; i++) {
            for (int j = 0; j < SEATS_PER_ROW; j++) {
                if (seats[i][j] == 0) {
                    System.out.print("O "); // Print 'O' for available seats
                } else {
                    System.out.print("X "); // Print 'X' for sold seats
                }
                if (j == 7) {
                    System.out.print("  "); // Gap between seats 8 and 9
                }
            }
            System.out.println("  £" + ROW_PRICES[i]); // Print the price of
the row
        }
    }

    private static void find_first_available() {
        for (int i = 0; i < ROWS; i++) {
            for (int j = 0; j < SEATS_PER_ROW; j++) {
                if (seats[i][j] == 0) {
                    System.out.println("First available seat: Row " + (i + 1)
+ " Seat " + (j + 1));
                    return;
                }
            }
        }
        System.out.println("No available seats."); // Message if no seats are
available
    }
```

```
    private static void print_tickets_info() {
        System.out.println("Sold Tickets:");
        double total = 0;
        for (int i = 0; i < ticketCount; i++) {
            System.out.println(tickets[i]);
            total += tickets[i].getPrice();
        }
        System.out.println("Total ticket sales: £" + total);
    }

    private static void search_tickets() {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter row number (1-3): ");
        int row = scanner.nextInt() - 1;
        System.out.print("Enter seat number (1-15): ");
        int seat = scanner.nextInt() - 1;

        for (int i = 0; i < ticketCount; i++) {
            if (tickets[i].getRow() == row && tickets[i].getSeat() == seat) {
                System.out.println(tickets[i]);
                return;
            }
        }
        System.out.println("Ticket not found.");
    }

    private static void sort_tickets() {
        Arrays.sort(tickets, 0, ticketCount,
Comparator.comparingInt(Ticket::getPrice));
        System.out.println("Tickets sorted by price:");
        print_tickets_info(); // Print sorted tickets
    }
}

// References (Sources I used to build this)
// www.youtube.com/@codeprolk - YouTube Channel
// www.youtube.com/@programmingwithmosh - YouTube Channel
// Google
// Open A
```

Ticket Class

```
import java.io.FileWriter;
import java.io.IOException;

public class Ticket {
    private int row; // Row number of the seat
    private int seat; // Seat number
    private int price; // Price of the ticket
    private Person person; // Person who bought the ticket

    public Ticket(int row, int seat, int price, Person person) {
        this.row = row;
        this.seat = seat;
        this.price = price;
```

```
        this.person = person;
    }

    // Getters and setters
    public int getRow() { return row; }
    public int getSeat() { return seat; }
    public int getPrice() { return price; }
    public Person getPerson() { return person; }


    public String toString() {
        return "Row " + (row + 1) + ", Seat " + (seat + 1) + ", Price £" +
price + ", " + person;
    }
}

//public void save() {
//    String file_name=row + seat + ".txt";
//    // print that details in text files
//    try {
//        //print ticket information
//        FileWriter writer = new FileWriter(file_name);
//        writer.write("Ticket information :\n");
//        writer.write("Row letter: " + row + "\n");
//        writer.write("Seat number: " + seat + "\n");
//        writer.write("Price: " + price + "\n");
//
//        //print person information
//        writer.write("Person Information: \n");
//        writer.write("Name is : " + person.getFirstName() + "\n");
//        writer.write("Surname is : " + person.getSName() + "\n");
//        writer.write("Email is : " + person.getEmail() + "\n");
//        writer.close();
//        System.out.println("Ticket information is in; " + file_name);
//    }catch(IOException e){
//        e.printStackTrace();
//
//    }
//}
```

Person Class

```
public class Person {
    private String name; // User's name
    private String surname; // User's surname
    private String email; // User's email

    public Person(String name, String surname, String email) {
        this.name = name;
        this.surname = surname;
        this.email = email;
    }

    // Getters and setters
    public String getName() { return name; }
    public String getSurname() { return surname; }
```

```
    public String getEmail() { return email; }


    public String toString() {
        return name + " " + surname + " (" + email + ")"; // String
representation of Person object
    }
}
```